

Relatório de Auditoria de Segurança

projeto **refract** — app desktop (Electron) + servidor de licenças

Data	11 de setembro de 2026
Escopo auditado	Raiz do repo (app Electron + React 19 + SQLite local), <i>lemonsqueezy-server/</i> (Fastify), <i>electron/</i> (main/IPC), <i>src/</i> (renderer), deploy (<i>Dockerfile</i> , <i>fly.toml</i> , <i>CI</i>). Excluídos: <i>node_modules</i> , <i>dist</i> , <i>release</i> .
Categorias	1) Banco sem tranca 2) Permissão no navegador 3) IDOR 4) Chaves expostas 5) Inputs/XSS
Metodologia	Somente achados verificados no código (arquivo:linha + trecho). Mapeamento por stack na nota abaixo; o que não se aplica é declarado N/A com motivo.

Nota metodológica — como cada categoria foi mapeada para a stack. O **refract** é um app **desktop single-user** (Electron 43 + React 19 + Vite; SQLite via *better-sqlite3* cru, sem ORM; auth por licença Ed25519 offline + trial local; deploy por *electron-builder* e CI GitHub; o único backend multiusuário é o *lemonsqueezy-server* em Fastify). Assim: **Cat. 1** foi avaliada no fator de posse *hwid* do servidor de licenças (o SQLite desktop é N/A por design — sem coluna de dono, isolamento via *userData* do SO); **Cat. 2** cruzou cada gate *isPremium/isTrial* do *renderer* com o enforcement no *main*; **Cat. 3** percorreu as 5 rotas Fastify + ~215 canais IPC + loopbacks; **Cat. 4** varreu código, configs, Docker/CI e histórico git; **Cat. 5** verificou sanitização no *renderer* e nos e-mails/templates.

Total de achados: **14** — ■ **Crítica: 2** ■ **Alta: 4** ■ **Média: 4** ■ **Baixa: 2** ■ **Informativa: 2**

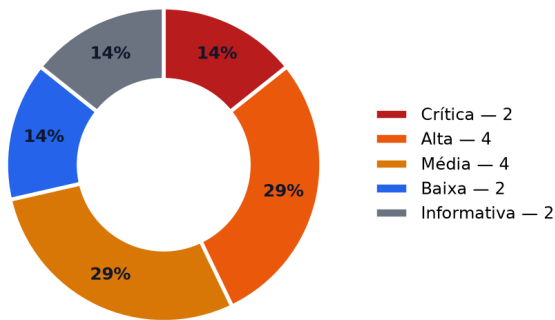
Detalhes, evidências linha a linha e 6 issues prontas para o GitHub nas seções seguintes.

1 Resumo executivo

Quatro riscos centrais: **(i)** a licença paga vaza por ID enumerável no modo padrão do servidor (F-04); **(ii)** o processo main executa shell e apaga arquivos com input do renderer (F-03, F-06) e lê arquivos arbitrários via LLM (F-05); **(iii)** chaves de billing reais dormem em texto claro no workspace (F-01, F-02); **(iv)** fronteiras locais confiam em quem chegar primeiro (F-09, F-10). Em compensação, webhook, lookup de licença, gates premium e sanitização XSS estão corretos — ver §3.

Severidade	Qtd	Leitura
Crítica	2	ação imediata (rotação de segredos).
Alta	4	correção no próximo ciclo; exploráveis com pré-requisito local.
Média	4	endurecimento programado.
Baixa	2	higiene / defesa em profundidade.
Informativa	2	sem risco direto; agrupada nas issues.

Achados por severidade (total 14)



Achados por categoria (F-04 conta na Cat. 1; também é IDOR)

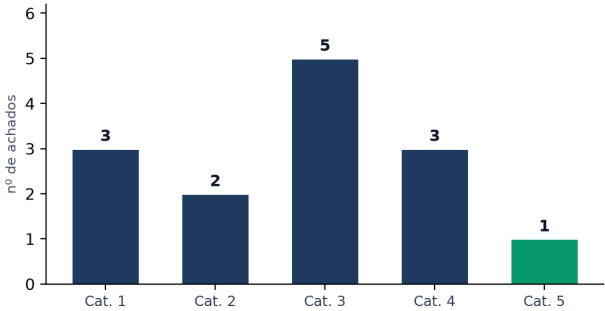


Figura 1 — distribuição por severidade e por categoria (Cat. 1 = Banco sem tranca · Cat. 2 = Permissão no navegador · Cat. 3 = IDOR · Cat. 4 = Chaves expostas · Cat. 5 = Inputs/XSS). F-04 é contabilizado na Cat. 1 (isolamento) e discutido também como IDOR.

2 Stack detectada

Camada	Deteccção / evidência
Linguagem	TypeScript + Node ≥ 20 (CI com Node 24).
Frontend	React 19 + Vite 5 + Tailwind (src/); renderer/ legado sem gates.
Backend desktop	Electron 43 (main) — ipcHandlers.ts + services + db locais.
ORM / queries	Nenhum — better-sqlite3 + SQL cru com prepared statements; sem Supabase/Prisma.
Auth	Licença Ed25519 offline (REFRACT-PRO) + trial local (safeStorage); sem JWT/sessão/RBAC.
Servidor remoto	lemonsqueezy-server (Fastify 5 + better-sqlite3) — único multiusuário.
Deploy	electron-builder (Win/macOS/Linux) + CI/release GitHub; LS via Dockerfile + fly.toml (gru, SQLite em volume). Sem Helm/Terraform/docker-compose.

3 Pontos fortes (verificado, com evidência)

- **Webhook íntegro e idempotente** — HMAC-SHA256 com *timingSafeEqual* + proteção de tamanhos diferentes (401 em vez de 500), idempotência por *event_key* e emissão única (*WHERE license_key IS NULL*) — *lemonsqueezy-server/server.js:145–154, 360–393*. Startup fail-fast sem segredos e sem chave privada (:72–89).
- **Self-service de licença exige dupla posse** — *POST /v1/license/lookup* só responde com **email + hwid exatos** (*server.js:340–356*) — saber só o e-mail não basta.
- **Confinamento de paths onde importa** — *delete-screenshot* e *analyze-image-file* rejeitam tudo fora de *userData* (*ipcHandlers.ts:454–462, 594–601*); uploads de perfil usam allowlist via diálogo nativo com TTL (:6195–6219); 3 geradores validam imagem (:5229, :5455, :5512).
- **XSS sob controle no frontend** — Zero *dangerouslySetInnerHTML* em *src/*; streaming sanitizado (:2580–2581); *ReactMarkdown* sem *rehype-raw*; renderer de markdown do phone com escape total; bloqueio de *javascript:* em URLs (*curlUtils.ts:207*); *open-external/mailto* com allowlist; *encodeURIComponent* no poll LS.
- **Premium verificado no main, não no navegador** — Todos os gates *isPremium/isTrial* da UI têm enforcement equivalente com *pro_required* no main (*modes:* :6721–7159, profile:* :6221–6627*); licença é criptografia Ed25519 offline verificada, não booleano do renderer.
- **Segredos locais bem guardados em runtime** — *CredentialsManager* criptografa com *safeStorage*, grava atômico (*tmp+rename*) e apaga o JSON legado; o *preload* expõe só booleanos *has*Key*, nunca a chave crua.
- **Cobertura real da auditoria** — 5 rotas Fastify + ~215 canais *safeHandle* + IPCs diretos (*RoleTwin, PurchaseActivation, LemonSqueezy, Keybinds, main.ts*) + 2 loopbacks HTTP (*Calendar :11111, PhoneMirror*) + estáticos *site/serve-all* — todos percorridos; sem RLS/Supabase/multi-tenant (N/A declarado).

4 Pontos fracos (riscos centrais)

- O ativo que gera receita (*license_key*) vaza por ID enumerável no modo padrão (F-04).
- O processo main executa shell com input do renderer (F-03), apaga arquivos por id cru (F-06) e lê arquivos arbitrários via LLM (F-05).
- Chaves de billing reais dormem em texto claro no workspace (F-01, F-02).
- Fronteiras locais (loopback OAuth, sender IPC) confiam em quem chegar primeiro (F-09, F-10).

5 Achados detalhados por categoria

Severidade, arquivo:linha exatos, descrição + por que é explorável. Trechos de segredos aparecem redigidos (prefixo + ...).

5.1 Categoria 1 — BANCO SEM TRANCA (isolamento de inquilino/dono) (3 achados)

Alta

F-04 Poll de licença devolve *license_key* sem exigir *hwid* (bypass legado ativo por padrão)

Arquivo:linha: *lemonsqueezy-server/server.js:65, 193–198, 281, 294–310*

O mecanismo de isolamento do servidor de licenças é o fator de posse **hwid** (*hwidAllowsAccess*). Com **LS_REQUIRE_HWID ≠ 1 (padrão)**, linhas de checkout sem *hwid* (clientes antigos gravam *hwid = NULL*) liberam a *license_key* para qualquer um que conheça o *checkout id* — IDs numéricos sequenciais do LemonSqueezy, enumeráveis; as respostas 404/403/200 viram oráculo. O próprio teste documenta o bypass (*server.test.mjs:65–66*) e o comentário :290–293 admite o risco.

Trecho:

```
const STRICT_HWID = process.env.LS_REQUIRE_HWID === '1'; // :65 default off
if (r && r !== 'unknown') return q === r; return strict ? q.length > 0 : true; // :193–198
const row = db.prepare('SELECT * FROM checkouts WHERE id = ?').get(id); // :296
if (row.license_key) return { license_key, status: 'paid', plan }; // :309–310
```

Média

F-07 POST /v1/checkout/lemonsqueezy sem rate-limit (spam/DoS de SQLite)

Arquivo:linha: lemonsqueezy-server/server.js:248-288 (vs :294, :340)

A criação de checkout é pública por design, mas é a única rota de escrita sem *rateLimitPreHandler*. Cada chamada insere uma linha *open* no SQLite e custa uma chamada à API do LemonSqueezy — spam intencional enche o disco/banco e gera custo.

Trecho:

```
app.post('/v1/checkout/lemonsqueezy', async (req, reply) => { // :248 sem preHandler
db.prepare('INSERT INTO checkouts ...').run(checkoutId, plan, ...); // :278-281
```

Baixa

F-12 Rate-limit em memória por IP, sem trustProxy; zero desativa; strict ainda aceita hwid qualquer

Arquivo:linha: lemonsqueezy-server/server.js:64, 161-212

O bucket é um *Map* em memória (morre no restart, não escala além de 1 instância, conta *req.ip* — falsificável via *X-Forwarded-For* sem *trustProxy*); *LS_RATE_LIMIT_PER_MIN=0* desativa. E mesmo com *STRICT_HWID=1*, linha legada aceita qualquer hwid não-vazio (*server.test.mjs:69* — *hwidAllowsAccess("", 'q', true) === true*), ou seja, não é prova de posse.

Trecho:

```
const RATE_LIMIT_PER_MIN = Number(process.env.LS_RATE_LIMIT_PER_MIN ?? 120); // :64
if (max <= 0) return { allowed: true, ... }; // :164 (0 = ilimitado)
```

5.2 Categoria 2 — PERMISSÃO DEFINIDA NO NAVEGADOR (2 achados)

Média

F-10 Canais IPC sem validação de remetente (qualquer janela invoca tudo)

Arquivo:linha: electron/ipcHandlers.ts:105-111 (vs :385-419)

safeHandle() só remove o handler anterior e registra — não valida *event.sender*. Apenas 2 handlers conferem *sender.id*.

Qualquer janela (launcher/overlay/settings) ou renderer comprometido via XSS pode invocar canais destrutivos (*flush-database :5051, delete-meeting :2208, set-*api-key*). Mitiga parcialmente: *contextIsolation + nodeIntegration:false* nas janelas.

Trecho:

```
const safeHandle = (channel, listener) => {
ipcMain.removeHandler(channel); ipcMain.handle(channel, listener); }; // :105-111
```

Informativa

F-14 Inconsistência de gate: RoleTwin exige premium puro; demais gates aceitam trial

Arquivo:linha: electron/services/RoleTwinManager.ts:224-226

O dossier do RoleTwin usa *isPremium()* puro (trial não libera), enquanto todos os demais gates usam *isProOrTrialActive()* (*ipcHandlers.ts:148-169*). Sem impacto de segurança — apenas inconsistência de produto a uniformizar.

Trecho:

```
if (!LicenseManager.getInstance().isPremium()) return null; // RoleTwinManager.ts:224-226
```

5.3 Categoria 3 — IDOR (objeto por ID sem checar dono) (5 achados)

Alta

F-03 Command injection via shell no GitService (filePath/branch/message do renderer)

Arquivo:linha: electron/services/GitService.ts:114-124, 207-208, 279, 348-349, 385, 397
git() interpola argumentos em shell (*execAsync(`git \${args}`)*). O escaping cobre apenas aspas duplas — *\$(...)*, backticks e *\$VAR* expandem dentro de "...". Os IPCs *git:diff / git:commit / git:create-branch / git:switch-branch / git:stash*

(*ipcHandlers.ts:7989-8034*) repassam strings cruas do renderer. Pré-requisito: renderer comprometido (XSS) ou janela maliciosa → **RCE no processo main**. O *commit* usa *git commit -F* - para a mensagem (correto), mas o *add -- "..."* anterior segue injetável.

Trecho:

```
return execAsync(`git ${args}`, { cwd: this.cwd, ... }); // GitService.ts:118
const target = filePath ? `-- "${filePath}"` : ''; // :207
await this.git(`checkout -b "${name.replace(/"/g, '\\"')}"`); // :385
```

Alta

F-05 gemini-chat / gemini-chat-stream aceitam imagePath sem validateImagePath

Arquivo:linha: electron/ipcHandlers.ts:610-622 e 714+ (vs :5229, :5455, :5512)

Os dois canais de chat repassam *imagePaths* direto ao LLM, sem a validação de confinamento a *userData* que outros 3 handlers aplicam (*curlUtils.ts:285-374*: *realpath + allowlist + bloqueio de symlink*). Renderer/XSS pode apontar para arquivos arbitrários (ex.: chaves, backups) e exfiltrar o conteúdo via respostas do LLM.

Trecho:

```
const result = await appState.processingHelper.getLLMHelper()
.chatWithGemini(message, imagePath, context, ...); // :620-622 sem validação
```

Alta**F-06** local-whisper-delete-model: traversal permite apagar fora do cache

Arquivo:linha: electron/ipcHandlers.ts:4344-4352 + electron/audio/whisper/modelManager.ts:90-92, 309-315
modelIdToCacheDir() devolve o id sem checar pertinência ao *MODEL_CATALOG*; o IPC repassa o id cru a *path.join(cacheDir, modelId) + rmSync(recursive)*. *modelId = ../../...* escapa do diretório de modelos e apaga dados arbitrários do usuário com privilégios do app.

Trecho:

```
function modelIdToCacheDir(modelId) { return modelId; } // modelManager.ts:90-92
const modelDir = path.join(cacheDir, modelIdToCacheDir(modelId)); // :311
fs.rmSync(modelDir, { recursive: true, force: true }); // :313
```

Média**F-08** repo-index:scan lê qualquer caminho do disco sem confinamento

Arquivo:linha: electron/ipcHandlers.ts:7557-7573 + electron/repo-indexer/RepoIndexer.ts:22-23, 56-75
O *repoPath* vindo do renderer é usado cru; *walkDir* faz *readDir/readFile* recursivo sem *realpath*, *allowlist* ou limite de tamanho/quantidade — leitura arbitrária de código-fonte/segregos + DoS de CPU/disco/DB vetorial. Condição de explorabilidade: flag *repoIndexer* ligada (*RepoIndexer.ts:42* — *scanRepo* retorna vazio se off).

Trecho:

```
safeHandle('repo-index:scan', async (_event, repoPath: string) => { // :7557
const indexer = new RepoIndexer({ repoPath, ... }); // :7560 sem validação
```

Média**F-09** OAuth do calendário em loopback sem state/PKCE (CSRF local)

Arquivo:linha: electron/services/CalendarManager.ts:88-127

O callback *http://localhost:11111/auth/callback* aceita *?code=* sem *state/nonce/PKCE* nem validação de *Origin*. Qualquer site aberto no navegador ou malware local pode chamar a URL com um *code* do atacante — fixação/login CSRF da conta Google conectada. Janela de exposição: 5 min por fluxo (:119-121).

Trecho:

```
const server = http.createServer(async (req, res) => { // :88
if (req.url?.startsWith('/auth/callback')) { // :90 sem state
server.listen(11111, ...); // :123 porta fixa
```

5.4 Categoria 4 — CHAVES EXPOSTAS (hardcode) (3 achados)

Crítica**F-01** Seis segredos reais de API/OAuth em .env na raiz do workspace

Arquivo:linha: .env:1-11 (raiz do projeto)

GOOGLE_CLIENT_SECRET (:2), GROQ_API_KEY (:3), OPENAI_API_KEY sk-proj (:4), GEMINI_API_KEY (:6), DEEPGRAM_API_KEY (:10) e ELEVENLABS_API_KEY (:11), além de path disclosure (:7). Chaves de billing: uso indevido gera custo direto; OAuth secret permite takeover de consentimento. Verificado: **não commitado** (*.gitignore:119*; *git ls-files* vazio; fora do histórico) — exposição é em disco/backup/sync/prints. Placeholders (:5, :12-15, :36) estão OK.

Trecho:

```
.env:2 GOOGLE_CLIENT_SECRET=GOCSPX-... (OAuth, crítica)
.env:3 GROQ_API_KEY=gsk_... | .env:4 OPENAI_API_KEY=sk-proj-...
.env:6 GEMINI_API_KEY=AQ.Ab8... | .env:10 DEEPGRAM... | .env:11 ELEVENLABS sk_...
```

Crítica**F-02** Service account GCP completa no workspace, referenciada em claro pelo .env

Arquivo:linha: tonal-history-500223-e0-a9ad29ad1df4.json:1-11

Arquivo com *private_key* (1,7 KB), *private_key_id*, *client_email* e *project_id* *tonal-history-500223-e0*; o *.env:7* aponta o caminho absoluto (vaza usuário Windows *daviif*). Quem lê o workspace assume o projeto GCP. Verificado: **não commitado** (*.gitignore:404*), mas sem criptografia em repouso.

Trecho:

```
"type": "service_account", // :2
"private_key": "-----BEGIN PRIVATE KEY-----...", // :5 (1679 chars)
"client_email": "sdas-372@...iam.gserviceaccount.com" // :6
```

Informativa**F-13** Chave privada Ed25519 de teste em disco

Arquivo:linha: lemonsqueezy-server/test-key.pem:1-3

Par de desenvolvimento citado no próprio *.gitignore* (:9 **.pem*) — fora do git (verificado). Risco apenas se *LICENSE_SIGNING_KEY_PATH* apontar para ela em produção; o default é *~/refract/license-signing-key.pem* (*server.js:62*) — OK hoje. Manter longe de prod.

Trecho:

```
-----BEGIN PRIVATE KEY----- // test-key.pem:1
MC4C...wmn // :2 (Ed25519 de teste)
```

5.5 Categoria 5 — INPUTS SEM TRATAMENTO (XSS) (1 achados)

Baixa

F-11 DOMPurify usado corretamente, mas é dependência transitiva não declarada

Arquivo:linha: src/components/RefractInterface.tsx:215, 2580-2581 + package.json (ausente) + src/types/vendor.d.ts:1

O único *innerHTML* com input do LLM é sanitizado (*marked.parse* → *DOMPurify.sanitize*), e o *ReactMarkdown* roda sem *rehype-raw* (HTML cru não executa) — **nenhum XSS explorável hoje**. A fragilidade: *dompurify* não está em *dependencies* (só transitiva via *package-lock.json:10053*, com shim em *vendor.d.ts:1*). Se a transitiva sumir, o build quebra — ou pior, alguém remove o import e o stream vira XSS fail-open.

Trecho:

```
import DOMPurify from 'dompurify'; // :215 (não declarada em package.json)
const rawHtml = marked.parse(streamingTextRef.current, { async: false }); // :2580
node.innerHTML = DOMPurify.sanitize(rawHtml); // :2581 OK
```

6 Recomendações priorizadas

Pri	Ação	Achados
P1	Rotacionar as 6 chaves do .env + a service account GCP; mover segredos para env do SO/keychain e .env.example só com placeholders; validação de startup que recuse placeholders em prod.	F-01, F-02
P2	Ligar LS_REQUIRE_HWID=1 em prod; exigir o hwid da criação para TODAS as linhas (migração das legadas); rate-limit no POST /v1/checkout; trustProxy; checkout ids opacos.	F-04, F-07, F-12
P3	Eliminar shell no GitService: execFile/spawn com argv + allowlist de filePath/branch; confinar set-cwd.	F-03
P4	validateImagePath em gemini-chat/stream; allowlist MODEL_CATALOG no whisper-delete; confinar repo-index:scan (realpath + limites).	F-05, F-06, F-08
P5	OAuth com state/PKCE + porta efêmera + check de Origin; validar event.sender nos IPCs sensíveis.	F-09, F-10
P6	Declarar dompurify em dependencies; confinar test-key.pem ao dev; uniformizar gate do RoleTwin.	F-11, F-13, F-14

7 Issues para o GitHub (texto pronto para copiar e colar)

Cada issue está entre marcadores --- ISSUE n --- e --- FIM ISSUE n ---. Achados triviais relacionados foram agrupados para evitar spam (6 issues para 14 achados).

--- ISSUE 1 ---

[Segurança] Segredos reais em disco: .env + service account GCP + chave de teste
Labels sugeridas: `security`, `severity:critical`
Descrição do problema e por que é explorável
O workspace contém segredos reais em texto claro: 6 chaves de API/OAuth no `.env` da raiz e uma service account GCP completa (`private_key` de 1,7 KB). Qualquer leitura do diretório (backup, sync em nuvem, screenshot, suporte remoto, infostealer) compromete billing (Groq/OpenAI/Gemini/Deepgram/ElevenLabs), consentimento OAuth e o projeto GCP. O `.env:7` ainda referencia a service account por caminho absoluto. A `test-key.pem` é de teste, mas é chave privada em disco.
Verificado: nenhum dos três está no git (`.gitignore:119`, `:404`, `lemonsqueezy-server/.gitignore:9`; `git ls-files` vazio; fora do histórico) – o risco é disco/backup, não histórico público.
Evidência

.env:1 GOOGLE_CLIENT_ID=1858... (ID público, enumera projeto)
.env:2 GOOGLE_CLIENT_SECRET=G0CSPX-... (OAuth, CRÍTICO)
.env:3 GROQ_API_KEY=gsk-... (billing)
.env:4 OPENAI_API_KEY=sk-proj-... (billing, 164 chars)
.env:6 GEMINI_API_KEY=AQ.Ab8... (billing)
.env:7 GOOGLE_APPLICATION_CREDENTIALS=C:\Users\...\tonal-history-....json
.env:10 DEEPGRAM_API_KEY=10d5... (40 hex)
.env:11 ELEVENLABS_API_KEY=sk_9e78...

tonal-history-500223-e0-a9ad29ad1df4.json:2-11 (type/PRIVATE KEY/client_email)
lemonsqueezy-server/test-key.pem:1-3 (Ed25519 de teste)

Impacto
- Uso indevido de APIs pagas (custo direto) e abuso de quotas.
- Takeover de consentimento OAuth Google; acesso ao projeto GCP via service account.
- Assinatura de licenças se a chave real vaziar por vizinhança (a de teste não assina prod hoje).
Sugestão de correção
1. Rotacionar IMEDIATAMENTE as 6 chaves e a service account (revogar as antigas).
2. Remover segredos do <code>`.env`</code> da raiz; usar env do SO/keychain; commitar só <code>`.env.example`</code> .
3. Mover a service account para fora do repo com ACL restrita (ou workload identity).
4. Garantir que <code>`LICENSE_SIGNING_KEY_PATH`</code> nunca aponte para <code>`test-key.pem`</code> em prod.
5. Validação de startup que recuse placeholders (<code>`your_*</code>) em produção.
Critérios de aceite
[] Chaves antigas revogadas nos provedores.
[] <code>`grep`</code> por <code>`sk-proj gsk_ GOCSPX BEGIN PRIVATE`</code> retorna só fixtures de teste.
[] <code>`.env`</code> sem valores reais; <code>`.env.example`</code> só com placeholders.
[] Service account fora do workspace (ou permissão mínima + rotação).
[] CI falha se placeholder for usado em prod.

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

[Segurança] Command injection no GitService via interpolação em shell (RCE pelo renderer)
Labels sugeridas: <code>`security`</code> , <code>`severity:high`</code>
Descrição do problema e por que é explorável
<code>`GitService.git()`</code> executa <code>`execAsync(`git \${args}`)`</code> — shell real. O escaping cobre só aspas duplas, mas <code>`\$(...)`</code> , backticks e <code>`\$VAR`</code> expandem dentro de <code>`"..."`</code> . Os canais <code>`git:diff`</code> , <code>`git:commit`</code> , <code>`git:create-branch`</code> , <code>`git:switch-branch`</code> , <code>`git:stash`</code> e <code>`git:set-cwd`</code> repassam strings cruas do renderer. Pré-requisito: renderer comprometido (XSS) ou janela maliciosa → RCE no processo main.
Evidência

electron/services/GitService.ts:114-124
private async git(args: string) {
return execAsync(`git \${args}`, { cwd: this.cwd, ... });
}
electron/services/GitService.ts:207-208
const target = filePath ? `-- "\${filePath}"` : ''; // \$(...) ainda expande
electron/services/GitService.ts:279, 348-349, 385, 397 (mesmo padrão)
electron/ipcHandlers.ts:7989-8034 (git:diff/commit/create-branch/switch-branch/stash/set-cwd)

PoC conceitual (NÃO executar em prod): <code>`filePath = 'a\$(calc).txt'`</code> → o shell avalia <code>`\$(calc)`</code> .
Impacto
- Execução arbitrária de comandos com privilégios do usuário; leitura/escrita total de arquivos.
- Transforma qualquer XSS futuro em comprometimento total do host.
Sugestão de correção
1. Trocar <code>`execAsync(string)`</code> por <code>`execFile('git', argv)`</code> / <code>`spawn`</code> sem shell em todos os métodos.
2. Validar <code>`filePath`</code> contra <code>`git ls-files`</code> + confinamento ao <code>`cwd`</code> ; branch por allowlist (<code>`^[A-Za-z0-9/_.-]+\$`</code>) + <code>`git check-ref-format`</code> .
3. Confinar <code>`set-cwd`</code> a raízes permitidas.
Critérios de aceite
[] Nenhum <code>`execAsync`/`execSync`</code> com template string contendo input em <code>`GitService.ts`</code> .
[] <code>`filePath='a\$(id).txt'`</code> e similares não executam nada (só erro de path inválido).
[] <code>`git:diff`</code> de arquivo válido continua funcionando.

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

[Segurança] Bypass de hwid no poll de licença + checkout sem rate-limit (endurecer lemonsqueezy-server)
Labels sugeridas: <code>`security`</code> , <code>`severity:high`</code>

Descrição do problema e por que é explorável
(a) Com `LS_REQUIRE_HWID≠1` (padrão), checkouts sem hwid (clientes antigos gravam NULL) liberam a `license_key` para quem conhecer o checkout id – IDs LS sequenciais e enumeráveis, com oráculo 404/403/200. (b) `POST /v1/checkout` não tem rate-limit (as outras rotas têm) → spam de linhas `open` + custo de API. (c) Rate-limit em memória/por-IP sem `trustProxy` (spoof de XFF) e `=0` desativa; mesmo em modo estrito, qualquer hwid não-vazio libera linha legada.
Evidência
... lemonsqueezy-server/server.js:65 STRICT_HWID default off lemonsqueezy-server/server.js:193-198 hwidAllowsAccess (bypass: ('','',false)===true) lemonsqueezy-server/server.js:281 hwid null (linha legada nasce sem hwid) lemonsqueezy-server/server.js:294-310 GET /v1/checkout/:id/license devolve license_key lemonsqueezy-server/server.js:248 POST /v1/checkout sem preHandler (vs :294 e :340 com) lemonsqueezy-server/server.test.mjs:65-69 (bypass documentado em teste) ...
Impacto
- Vazamento de licenças pagas (bypass de pagamento) por enumeração de checkout ids. - DoS de disco/SQLite + custo de API LemonSqueezy via spam de checkouts.
Sugestão de correção
1. `LS_REQUIRE_HWID=1` em prod + migração: exigir o hwid do POST para TODAS as linhas. 2. `rateLimitPreHandler` no POST (+ captcha se o spam persistir). 3. `trustProxy` + limite distribuído (ou documentar single-instance); recusar `=0` em prod. 4. Considerar checkout ids opacos (UUID).
Critérios de aceite
[] `GET /:id/license` sem `?hwid=` correto retorna 403 para qualquer linha. [] Linha legada (`hwid NULL`) + `?hwid=qualquer` retorna 403. [] POST sob flood (>120/min/IP) retorna 429 sem inserir linhas. [] `server.test.mjs` atualizado cobrindo os 3 casos.

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

[Segurança] Validação de paths ausente em 3 IPCs: chat (leitura), whisper-delete, repo-scan
Labels sugeridas: `security`, `severity:high`
Descrição do problema e por que é explorável
Três canais confiam em path/id cru do renderer: (1) `gemini-chat/stream` repassa `imagePaths` ao LLM sem `validateImagePath` (leitura arbitrária + exfiltração via respostas); (2) `local-whisper-delete-model` faz `path.join(cache, modelId) + rmSync(recursive)` sem allowlist (traversal `../../` apaga dados); (3) `repo-index:scan` percorre `repoPath` arbitrário com `readdir/readFile` recursivo (leitura + DoS). O projeto JÁ tem o padrão correto – só não aplicado aqui.
Evidência
... electron/ipcHandlers.ts:610-622 gemini-chat → chatWithGemini(message, imagePaths...) sem validação electron/ipcHandlers.ts:714+ gemini-chat-stream idem (vs :5229, :5455, :5512 que validam) electron/ipcHandlers.ts:4344-4352 local-whisper-delete-model → deleteModel(modelId) cru electron/audio/whisper/modelManager.ts:90-92 modelIdToCacheDir retorna id; :309-315 join+rmSync electron/ipcHandlers.ts:7557-7573 repo-index:scan → new RepoIndexer({ repoPath }) cru electron/repo-indexer/RepoIndexer.ts:22-23, 56-75 walkDir recursivo sem confinamento ...
Impacto
- Leitura de arquivos sensíveis e exfiltração via LLM; destruição de dados; DoS de CPU/disco.
Sugestão de correção
1. Chamar `validateImagePath()` em `gemini-chat/stream` (rejeitar fora de userData). 2. `deleteModel`: allowlist `id ∈ MODEL_CATALOG` + `path.resolve` contido no cache. 3. `repo-index:scan`: `realpath` + confinamento a raízes configuradas + limite de arquivos/tamanho.
Critérios de aceite
[] `imagePaths` fora de userData rejeitado com erro. [] `deleteModel('../../x')` e id fora do catálogo rejeitados; catálogo válido ok. [] `repo-index:scan` fora das raízes rejeitado. [] Testes de regressão espelhando `validateImagePath.test.mjs`.

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

[Segurança] CSRF no OAuth loopback do calendário + IPCs sem checagem de remetente
Labels sugeridas: `security`, `severity:medium`
Descrição do problema e por que é explorável
(a) O callback OAuth `http://localhost:11111/auth/callback` aceita `?code=` sem `state/PKCE` nem Origin – site malicioso ou processo local injeta `code` do atacante (login CSRF/fixação, janela de 5 min). (b) `safeHandle()` não valida `event.sender`; só 2 handlers conferem remetente – qualquer janela/XSS invoca canais destrutivos (`flush-database`, `delete-meeting`, troca de API keys).
Evidência
``` electron/services/CalendarManager.ts:88-127 (loopback, :90 sem state, :123 porta fixa) electron/ipcHandlers.ts:105-111 safeHandle sem sender check (vs :385-419 que checam) electron/ipcHandlers.ts:5051 flush-database, :2208 delete-meeting (sem confirmação de origem) ```
<b>Impacto</b>
- Conexão da conta Google da vítima a credenciais do atacante; destruição de dados locais via IPC.
<b>Sugestão de correção</b>
1. OAuth: `state` aleatório + PKCE, validar `state`, porta efêmera (0) e check de Origin; timeout curto. 2. IPC: helper validando `event.sender` nos canais sensíveis; confirmação para `flush-database`.
<b>Critérios de aceite</b>
[ ] Callback com `state` errado/ausente rejeitado; fluxo válido funciona. [ ] Canal sensível de webContents não autorizado rejeitado. [ ] `flush-database` exige confirmação explícita.

## --- FIM ISSUE 5 ---

## --- ISSUE 6 ---

<b>[Segurança] Higiene: declarar dompurify + confinar chave de teste + uniformizar gate RoleTwin</b>
Labels sugeridas: `security`, `severity:low`
<b>Descrição do problema e por que é explorável</b>
Três higiene de baixo risco: (a) `dompurify` – que sustenta TODA a sanitização do stream do LLM – é dependência transitiva não declarada (se sumir, o build quebra ou a sanitização some); (b) `test-key.pem` é privada em disco (hoje fora de prod, mas sem trava); (c) RoleTwin usa `isPremium()` puro enquanto o resto usa `isProOrTrialActive()` (inconsistência trial). Nenhum é explorável hoje: zero XSS ativo, default da chave correto, gates premium enforced no main.
<b>Evidência</b>
``` src/components/RefractInterface.tsx:215, 2580-2581 (import + uso correto) package.json (dompurify ausente) vs package-lock.json:10053 + src/types/vendor.d.ts:1 lemonsqueezy-server/test-key.pem:1-3 + server.js:62 (default correto) electron/services/RoleTwinManager.ts:224-226 vs ipcHandlers.ts:148-169 ```
Impacto
- Regressão futura: transitiva removida reabre XSS no stream; assinatura com chave de teste invalidaria licenças; UX inconsistente no trial.
Sugestão de correção
1. `npm i -S dompurify` (+ `@types/dompurify` em dev); teste que falha se `sanitize` sumir do bundle. 2. Mover `test-key.pem` para `fixtures/` com README dev-only + trava anti-prod. 3. RoleTwin para `isProOrTrialActive()` (ou documentar a exceção).
Critérios de aceite
[] `dompurify` em `dependencies`; build sem hoisting ambíguo. [] Fumaça: `  ` no stream não executa. [] Nenhum caminho de prod referencia `test-key.pem`. [] Gate do RoleTwin documentado/testado.

--- FIM ISSUE 6 ---